

Sockets

¿Qué es un Socket?

Un socket es un punto de comunicación entre dos programas que permite el intercambio de datos en una red. Es una interfaz que conecta una aplicación con la pila de red del sistema operativo, utilizando protocolos como TCP o UDP.

Características:

Dirección IP y Puerto: La IP identifica un dispositivo y el puerto, una aplicación en ese dispositivo.

Protocolos:

- TCP/IP:
 - Orientado a la conexión
 - Los paquetes se reciben TODOS y en ORDEN
 - Se envía un "acknowledge" para cada paquete recibido
 - Se valida cada paquete mediante un Checksum
 - Los datos pueden ser transmitidos simultáneamente en ambas direcciones
 - "ai.socktype=SOCK_STREAM"
- UDP:
 - Sin conexión (Datagramas). Si Multicast/Broadcast
 - Los datagramas pueden perderse o llegar duplicados o fuera de secuencia; pero si llegan llegan completos.
 - Se utiliza un Checksum pero sólo para descartar paquetes (no hay retransmisión)
 - Los datos pueden ser transmitidos simultáneamente en ambas direcciones
 - "ai.socktype=SOCK_DGRAM"

Tipos:

- Stream socket (SOCK_STREAM): Para comunicación fiable (TCP).
- Datagram socket (SOCK_DGRAM): Para comunicación rápida y sin conexión (UDP).
- Raw socket: Acceso directo al nivel IP, usado en casos avanzados.

Ejemplo básico:

1. El servidor crea un socket y escucha en un puerto.
2. El cliente crea un socket y se conecta al servidor.
3. Ambos intercambian datos una vez conectados.

Funciones:

ntohs (Network TO Host Short): Convierte un valor de 16 bits desde el formato de byte order de red (big-endian) al formato de byte order del host.

Uso típico: Leer números de puerto o datos recibidos de la red.

htons (Host TO Network Short): Convierte un valor de 16 bits desde el formato de byte order del host al formato de byte order de red (big-endian).

Uso típico: Preparar números de puerto o datos para enviarlos por la red.

Son fundamentales en el uso de **sockets** porque garantizan que los datos enviados y recibidos entre diferentes sistemas sean interpretados correctamente, sin importar la arquitectura del hardware (big-endian o little-endian).

Ejercicio:

7) Escriba un programa que reciba por **línea de comandos un Puerto y una IP**. El programa debe aceptar una **única conexión e imprimir en stdout la sumatoria de los enteros recibidos en cada paquete**. Un paquete está definido como una **sucesión de números recibidos como texto, en decimal, separados por comas y terminado con un signo igual** (ej: "27,12,32="). Al recibir el texto 'FIN' debe **finalizar el programa ordenadamente** liberando los recursos.

```
int main(int argc, char *argv[]) { // ./server localhost 8080
    if (argc != 3) return -1;

    char *ip = argv[1];
    char *puerto = argv[2];

    struct addrinfo hints{}, *addr;
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_flags = AI_PASSIVE;

    int res = getaddrinfo(ip, puerto, &hints, &addr);
    if (res != 0) return -1;

    int skt = socket(addr->ai_family, addr->ai_socktype, addr->ai_protocol);
    if (skt < 0) {
        freeaddrinfo(addr);
        return -1;
    }

    res = bind(skt, addr->ai_addr, addr->ai_addrlen);
    if (res < 0) {
        freeaddrinfo(addr);
        close(skt);
        return -1;
    }

    freeaddrinfo(addr);

    res = listen(skt, 1);
    if (res < 0) {
        close(skt);
        return -1;
    }

    int client = accept(skt, nullptr, nullptr);
    if (client < 0) {
        close(skt);
        return -1;
    }
}
```

```

char c;
std::string numero;
int valor_total = 0;
while (true) {
    ssize_t bytes = recv(client, &c, sizeof(c), 0);
    if (bytes < 0) break;

    if (c == '=') {
        std::cout << valor_total << std::endl;
        valor_total = 0;
        numero = ""; // limpio el número
    } else if (c == ',') {
        try {
            valor_total += std::stoi(numero); // convertir a int y sumar
        } catch (const std::invalid_argument&) {
            std::cerr << "Valor no válido para convertir a entero" << std::endl;
        }
        numero = ""; // limpio el número
    }

    numero += c;

    if (numero.size() >= 3) {
        std::string palabra = numero.substr(numero.size() - 3);
        if (palabra == "FIN") break;
    }
}

close(client);
shutdown(skt, SHUT_RDWR);
close(skt);

return 0;
}

```

Desarrollo:

1. Definir variables

struct addrinfo hints{}, *addr : Se define la estructura que contiene información sobre la conexión de red. En este caso:

- ai_family = AF_INET : Define que se utilizará IPv4.
- ai_socktype = SOCK_STREAM : Define que se usará un socket de tipo TCP (stream).
- ai_flags = AI_PASSIVE : Indica que el socket debe ser usado para un servidor (escuchar conexiones entrantes).

2. Obtener la info de la dir

int res = getaddrinfo(ip, puerto, &hints, &addr) : resuelve el nombre de host y el puerto a una estructura addrinfo que contiene detalles sobre cómo conectarse

3. Crear el socket

int skt = socket(addr->ai_family, addr->ai_socktype, addr->ai_protocol) : socket() crea un descriptor de socket con los parámetros obtenidos de getaddrinfo()

4. **Bind socket**

`res = bind(skt, addr->ai_addr, addr->ai_addrlen)` : `bind()` asocia el socket a la dirección IP y puerto especificados

5. **Liberar mem**

`freeaddrinfo(addr)`

6. **Escuchar conexiones**

`res = listen(skt, 1)` : `listen()` pone el socket en modo de escucha, lo que significa que está esperando conexiones entrantes. El número 1 es el tamaño de la cola de conexiones pendientes.

7. **Aceptar una conexión**

`int client = accept(skt, nullptr, nullptr)` : `accept()` acepta una conexión entrante desde un cliente. Si la aceptación es exitosa, se devuelve un nuevo descriptor de socket `client`

8. **Lógica del ejercicio**

9. **Cerrar los sockets**

`close(client);`

`shutdown(skt, SHUT_RDWR);`

`close(skt);`

Protocolos

Protocolos y formatos

Protocolos en Texto: Más fáciles de debuggear, independientes del endianness y padding, pero lentos, ineficientes y más difíciles de parsear. Dependen del encoding del texto y de los delimitadores.

Protocolos en Binario: Más rápidos y eficientes en memoria y procesamiento, pero más difíciles de debuggear. Requieren considerar endianness, padding, tamaños y signos.

Endianness: Es el orden de los bytes en memoria para datos multibyte:

Big-endian: El byte más significativo (MSB) va primero. Ejemplo: `0x12345678` se almacena como `12 34 56 78`.

Little-endian: El byte menos significativo (LSB) va primero. Ejemplo: `0x12345678` se almacena como `78 56 34 12`.

La diferencia de endianness puede causar problemas de compatibilidad en la transmisión de datos binarios.

Padding: Es la inserción de bytes extra para alinear datos en memoria, mejorando el acceso en ciertas arquitecturas.

Ejemplo: En C, un entero (4 bytes) seguido de un carácter (1 byte) puede incluir 3 bytes de padding para alinear la siguiente variable en una dirección múltiplo de 4. Esto puede alterar el tamaño al serializar los datos.

Ejemplos: HTTP, TVL

Ejemplos:

7) Considere la estructura `struct ejemplo { int a; char b;};`. ¿Es verdad que `sizeof (ejemplo)=sizeof(a) +sizeof(b)`? Justifique.

No, no es verdad ya que `sizeof(a) = 4` y `sizeof(b) = 1` pero `ejemplo = 8`. Esto se debe al padding, que sirve para alinear los datos en memoria. Por lo que, `ejemplo = 4 + 1 + 3` (padding).

Ejemplo en un sistema de 64 bits:

`char` → 1
`short` → 2
`int` → 4
`long` → 8
`long long` → 8
`float` → 4
`double` → 8
`long double` → 16
Punteros → 8

Struct y Clase

```
1 struct Vector {  
2     int *data; // public by default  
3     int size;  // public by default  
4 };  
5  
6 class Vector {  
7     int *data; // private by default  
8     int size;  // private by default  
9 };
```

Absolutamente todo lo visto con structs en C++ aplica a las clases de C++. La única diferencia es que las clases tienen sus atributos y métodos privados por default.

Ejercicios

8) Indique la **salida** del siguiente programa:

```
class A { A(){cout << "A()" << endl;} ~A(){ cout << "~A()" << endl;} }  
class B : public A { B(){cout << "B()" << endl;} ~B(){ cout << "~B()" << endl;} }  
int main () { B b; return 0;}
```

La salida del siguiente programa es:

A B B A

Ya que padre → hijo

A()

B()

~B()

~A()

RAII

RAII: Resource Acquisition Is Initialization

- La idea es simple: si hay un recurso (memoria en el heap, un archivo, un socket) hay que encapsular el recurso en un objeto de C++ cuyo constructor lo adquiera e inicialice y cuyo destructor lo libere.
- La clave está en el diseño simétrico del par constructor-destructor.
- Es la diferencia entre alguien que sabe C++ de alguien que escribe código que compila.
- Al instanciarse los objetos RAII en el stack, sus constructores adquieren los recursos automáticamente.
- Al irse de scope cada objeto se les invoca su destructor automáticamente y por ende liberan sus recursos sin necesidad de hacerlo explícitamente.
- El código C++ se simplifica y se hace más robusto a errores de programación: RAII + Stack es uno de los conceptos claves en C++.

RAII

Es una patrón de diseño que nos permite encapsular la adquisición de recursos y su posterior liberación en un solo objeto, nos permite tener implementaciones más seguras, ya que al irse de scope cada objeto se les invoca su destructor automáticamente y por ende liberan sus recurso sin necesidad de hacer explícitamente.

Ejercicios:

9) ¿Qué ventaja ofrece un **lock raai** frente al tradicional **lock/unlock**?

RAII (Resource Acquisition Is Initialization) es un patrón de diseño que permite una gestión de la memoria más segura y eficiente, por ejemplo un lock raai frente a un lock tradicional va a llamar a su destructor de forma implícita y automática al salir del scope donde se declaró. Este tipo de locks nos ayuda a evitar errores tales como no olvidarse de llamar a unlock() cayendo en un posible deadlock.

8) ¿En qué consiste el patrón de diseño **RAII**? Ejemplifique.

Consiste en que si hay un recurso (memoria en el heap, un archivo, un socket) hay que encapsular el recurso en un objeto de C++ cuyo constructor lo adquiera e inicialice y cuyo destructor lo libere.

```
struct Buffer {
    Buffer(int size) {
        this->data = new char[size];
    }
    ~Buffer() {
        delete[] this->data;
    }
};

struct File {
    File(const char *name, const char *flags) {
        this->f = fopen(name, flags);
        if (!this->f) throw std::exception("fopen failed");
    }
    ~File() {
        fclose(this->f);
    }
};
```

Const y Member Initialization List

Constantes

- `const int` y `int const` son equivalentes así como también `const int *` y `int const *`. Sin embargo es distinto `int * const`.
- `const int * p` se lee como "p es un puntero; a int; constante" mientras que `int * const p` se lee como "p es constante; puntero; a int". El primero apunta a ints constantes mientras que el segundo es el puntero quién es constante.

Member Initialization List

- La member initialization list es el único lugar para inicializar atributos constantes y referencias.

Pasaje y Asignación de objetos (Move Semantics)

Pasaje por referencia

- En C++ podemos usar el pasaje por referencia. Una referencia es como un alias del objeto referenciado.
- Las referencias funcionan como un alias y el compilador en algunos casos ni siquiera reservará memoria para una referencia.

Pasaje por copia

- En C y en C++ el pasaje por default es por copia: cuidado de hacer una copia sin intención, puede traer un comportamiento inesperado
- La copia tanto en C como en C++ es bit a bit y funciona bien para objetos simples.
- Pero cuando hay punteros, la copia es del puntero y no del valor apuntado: la copia es superficial y no en profundidad (deep copy).
- Con 2 objetos apuntando al mismo heap, al destruirse uno libera el heap dejando al segundo objeto apuntando a la nada (use after free).
- *Evitar a toda costa las copias, son la principal causa de ineficiencias en código C y C++.*

Constructor por Copia

- Para crear un objeto nuevo a partir de otro se invoca al constructor por copia.
- Como cualquier otro constructor, el constructor por copia tiene una member initialization list para pasarle argumentos a los constructores de sus atributos.
- Todos los objetos en C++ son copiables por default. Si un objeto no tiene un constructor por copia, C++ le creará un constructor por copia por default que implementa una copia bit a bit naive. Por esta razón es muy fácil que un objeto se copie sin querer, algo que es difícil de debuggear.

Pasaje por movimiento: Move semantics

Ownership → A diferencia de una copia, el constructor por movimiento le roba o mueve los atributos del objeto fuente.

```
1 struct Vector {
2     int *data;
3     int size;
4
5     Vector(Vector&& other) {
6         this->data = other.data;
7         this->size = other.size;
8
9         other.data = nullptr;
10        other.size = 0;
11    }
12
13    ~Vector() {
14        if (data)
15            free(data);
16    }
17 };
```

- Cómo se implementa la transferencia del ownership dependerá de cada objeto. En este caso, al poner el puntero `data = nullptr` indicamos que no tiene más el ownership del recurso y por lo tanto no tiene que destruirlo.

Asignación por copia

- Todos los objetos en C++ son copiables por asignación así que si un objeto no implementa la sobrecarga del operador asignación, C++ le creará una implementación por default que hara una copia bit a bit naive.

Asignación por movimiento

```

10 void swap(Vector& a, Vector& b) {
11     Vector t = std::move(a); // a se mueve a t (constructor)
12     a = std::move(b); // b se mueve a a (asignacion)
13     b = std::move(t); // t se mueve a b (asignacion)
14 }

```

Objetos NO copiables

```

1 struct File {
2     public:
3     File copy(const char *to_where) { ... }
4
5     private:
6     File(const File &other) = delete;
7     File& operator=(const File &other) = delete;
8
9 };

```

PUNTOS IMPORTANTES (Segun DIPA):

- En el 99% de los casos, no quieres que tus clases sean copiables (sea por que no tiene sentido o por performance). Deshabilitar el constructor y operado asignación por copia.
- Implementar siempre move semantics (constructor y operador asignación).
- Usar `const` en donde sea posible.

Ejercicios

1) Explique qué es y para qué sirve un **constructor de copia** en C++. a) Indique cómo se comporta el sistema si éste **no es definido por el desarrollador**; b) Explique al menos una estrategia para **evitar que una clase particular sea copiable**; c) Indique qué **diferencia** existe entre un constructor de **copia** y uno **move**.

Un constructor de copia en C++, es un tipo de constructor que permite crear un nuevo objeto como copia de uno ya existente y sirve para definir cómo se copia un objeto.

- Si a una clase no le definimos un constructor de copia, C++ creará un constructor de copia por default que implementa una copia bit a bit naive

b. Public delete:

```
Clase(const Clase &c1) = delete;  
Clase &operator=(const Clase &c1) = delete;
```

- c. La diferencia principal es que en el constructor move si bien creamos un objeto a partir de otro, este constructor le roba o mueve los atributos del objeto fuente dejando a este en un estado válido pero indefinido ya que pierde el ownership de los atributos, a su vez optimiza el rendimiento porque evita crear copias innecesarias.

3) **Describa con exactitud las siguientes declaraciones/definiciones globales:**

1. `void (*F)(int i);`
2. `static void B(float a, float b){}`
3. `int *(*C)[5];`

1. F es un puntero a una función void que recibe como parámetro un entero.
2. La firma de una función llamada B de tipo void que recibe dos parámetros de tipo float. Cómo está declarada con el modificador static, su alcance está limitado al archivo en el que está definida.
3. C es un puntero a un arreglo de 5 elementos donde cada uno es un puntero a un número entero.

1) Explique breve y concretamente qué es f:

```
int (*f)(short int *, char[3]);
```

f es un puntero a una función que retorna un int que como parámetros recibe un puntero a un short int y un arreglo de 3 caracteres.

Threads

C + +, desde su estándar C++11 introduce el objeto `std::thread` a la STL.

Para lanzar un thread, basta con construir un objeto `std::thread` y pasarle algo "llamable".

Mientras el hilo principal no llama a `join()` hay dos secuencias independientes de ejecución, que **corren virtualmente en paralelo**.

Acceso Concurrente

- Hay un recurso compartido
- Hay más de un thread accediendo al recurso
- Hay operaciones de escritura (al menos una)
- Hay operaciones no atómicas

Mutex y Critical Sections

C++ nos provee un mecanismo de sincronización llamado "mutex". Un mutex es como un semáforo que solamente toma valor 0 o 1. Y nos va a servir para delimitar regiones críticas, o critical sections.

A esos cachitos de código que nos gustaría que sean atómicos los tenemos que detectar a mano, y los vamos a llamar "Critical sections".

Cuando los detectemos, los vamos a delimitar con un mutex. Al arrancar la CS vamos a usar el método `lock()`, y al finalizar el método `unlock()`

Tiene que existir UN mutex por recurso.

La STL incluye una clase `RAII` para manejar los mutex, que se llama `lock_guard`, y sirve para hacer lo mismo, pero sin "olvidarnos" del `unlock`

Monitores

- Al recurso lo pensamos (y codificamos) como si no hubiera problemas de concurrencia: el recurso NO CONOCE AL MUTEX
- Creamos una clase "Monitor" con el mutex y el recurso como atributos
- Agregamos al monitor un método por cada Critical Section que hayamos detectado
- LISTO! Hay una clase que tiene la lógica para sincronizar y otra con la lógica de negocio, no se tocan, bajo acoplamiento, alta cohesión

Problemas

- Context switch
 - Lanzar muchos hilos no siempre es lo más performante
 - Cambiar qué hilo está usando el procesador tiene un costo
 - Usar varios hilos tiene que estar justificado
- Deadlocks → dos o más hilos (o procesos) se quedan esperando indefinidamente porque cada uno necesita un recurso que está siendo bloqueado por otro hilo
- Starvation → es más difícil de detectar, pero es cuando un hilo no ejecuta sus operaciones
- Contención
 - Las Critical Sections están mal delimitadas, el programa es thread-safe, pero el mutex está tomado "demasiado tiempo"
 - El Monitor que vimos no es la técnica de programación concurrente con menos contención, pero es la más fácil de entender y estamos aprendiendo

Recursos compartidos

- Una **race condition** se debe al acceso no-atómico de lecto/escritura de un recurso compartido.
- Si el recurso compartido es inmutable o solamente se accede a él para operaciones de lectura, no existe la posibilidad de tal error.

Sincronización

- Para evitar la race condición debemos hacer que los hilos se coordinen entre sí para evitar que accedan al objeto compartido a la vez.
- Existen varias estrategias de sincronización: semáforos, colas, sockets (estos últimos también usados para comunicación)
- Un mutex es un objeto que nos permitirá forzar la ejecución de un código de forma exclusiva por un hilo a la vez. En C++ `std::mutex`
 - El uso del mutex puede verse como una variable booleana. La instrucción `std::mutex::lock()` setea a 1 si la variable es 0 de forma atómica mientras que `std::mutex::unlock()` setea la variable a 0 de forma atómica.

Ejercicios

- 5) ¿Cómo se logra que 2 **threads** accedan (lectura/escritura) a un mismo recurso compartido sin que se generen problemas de consistencia? **Ejemplifique.**

Se logra utilizando la sincronización. Debemos hacer que los hilos se coordinen entre sí para evitar que accedan al objetivo compartido a la vez. Existen varias estrategias pero voy a ejemplificar con el mutex:

Un objeto que nos permitirá forzar la ejecución de un código de forma exclusiva por un hilo a la vez.

```
int counter = 0;
std::mutex m;

void inc() {
    m.lock();
    ++counter;
    m.unlock();
}

int main(int argc, char* argv[]) {
    std::thread t1 {inc};
    std::thread t2 {inc};

    t1.join(); t2.join();
    return counter;
}
```

Para lograr que 2 threads accedan (lecto/escritura) a un mismo recurso compartido y evitar problemas tales como race conditions (que se debe al acceso no atómico de (lectura/escritura) de un recurso compartido), una forma conveniente de evitarlo es a través del uso de mutex, un objeto que nos permitirá (FECFE) Forzar la Ejecución de un Código de Forma Exclusiva por un hilo a la vez. Por supuesto, una vez tomado el mutex, el hilo debe soltarlo, de otra forma el recurso quedará inaccesible para el resto de los hilos, potencialmente llevando a una situación de deadlock

Queues Thread Safe

Conditional Variables:

Una condition variable es una primitiva de sincronización en C++ que, junto con un `std::mutex`, permite bloquear uno o más hilos hasta que otro hilo modifique una variable compartida (la condición) y notifique al `std::condition_variable`.

El hilo que modifica la condición debe:

1. Adquirir un `std::mutex` (usualmente con `std::lock_guard`).
2. Modificar la variable compartida mientras mantiene el bloqueo.
3. Llamar a `notify_one` o `notify_all` en el `std::condition_variable` (puede hacerlo tras liberar el mutex).

El hilo que espera debe:

1. Adquirir un `std::unique_lock<std::mutex>` para proteger la variable compartida.
2. Usar `wait`, `wait_for` o `wait_until` en el `std::condition_variable`. Esto libera el mutex mientras el hilo espera y lo recaptura al despertar.
3. Comprobar la condición tras despertar, ya que pueden ocurrir despertares espurios.
4. Las condition variables facilitan la sincronización entre hilos y garantizan un acceso seguro y eficiente a recursos compartidos.

¿Que tipos de colas vimos en Taller?

1. Cola Protegida: Thread safe, pero no bloqueante
2. Cola Bloqueante: `pop` solamente retorna cuando tiene elementos (o lanza exception cuando no hay más y la cola está cerrada)
3. Cola Bloqueante "Bounded": `push` también espera a que haya espacio

Cuándo vamos a usar alguna de estas?

Si vamos a tener un hilo que solamente consume, necesitamos algo bloqueante en ese hilo (recordar, patrón común, no obligatorio: 1 hilo - 1 operación blocking)

Si nos preocupa que no crezca indefinidamente, vamos a usar una bounded, pero ojo! Estamos metiendo otra operación bloqueante...

Y la cola protegida la vamos a usar cuando tenemos multithread, pero después de consumir los elementos queremos hacer algo más

Problema de productor/consumidor:

2 actores → Productor: crea objetos y los mete en una estructura FIFO

→ Consumidor: saca los objetos de la estructura y los procesa.

La estructura del medio (cola) es un recurso compartido, al acceder varios hilos y hacer `push` & `pop`, tenemos todo lo necesario para tener una race condition.

COMO SOLUCIÓN → a estas acciones/métodos (que conformar la critical section) los vamos a implementar en un monitor

Con esto solo la estamos haciendo una cola protegida, para hacerla bloqueante hace falta hacer uso de una `condition_variable` que nos permita avisar cuando se va a cumplir una condición o no, de esta forma bien podríamos pasar a tener una cola bloqueante, que solo haga `pop` en casos donde tengamos elementos. (a su vez evitamos una busy waiting ya que un error podría ser verificar que no se cumpla una condición y llamar a `sleep`).

A su vez si quisieramos evitar que la cola crezca indefinidamente podríamos hacer uso de una conditional variable para permitir que solo se haga push cuando haya espacio en la cola: a esta cola se la conoce como cola bloqueante → 'bounded'
(Usando una cv evitamos una busy waiting ya que un error podría ser verificar que no se cumpla una condición y llamar a sleep) → defecto de la cola protegida al no usar cv.

Dato de color:

Cuando usamos una cv, vamos a hacer uso de un while debido al spurious wake-up:

```
while(condition){  
    cv.wait(lock); // preferiblemente un unique_lock  
}
```

ya que podría ocurrir que el wait salga sin que nosotros queramos , tales como que el hilo fue avisado, timeout o un "spurious wake-up"

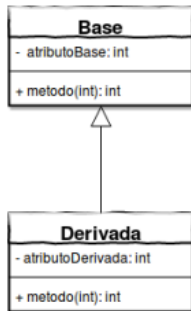
Ejercicios:

10) ¿Qué significa que una función es **blocante**? ¿Cómo subsanaría esa limitación en términos de **mantener el programa 'vivo'** ?

Una función bloqueante detiene la ejecución del programa hasta completar su tarea. Un ejemplo es la función accept del socket del profe que usamos durante la cursada, donde la función "espera" una conexión. Para subsanar esta limitación podemos hacer uso de threads, lo que nos va a permitir realizar tareas concurrentes dentro de un mismo proceso.

Herencia y Polimorfismo

Herencia



¿Qué es la herencia?

1. Es un concepto de la OOP
2. Establece una relación "es un"
3. Se dice que:
 - 3.1 Una clase derivada extiende a una clase base.
 - 3.2 Una clase base generaliza varias clases derivadas

Herencia en C++

En C++ heredamos indicando un tipo de herencia (usaremos public por default)

```
class Derivada: public Base { ... }
```

No se heredan: Los constructores, Los operadores, friend

También se puede heredar en forma protected o private.

¿Como se construye Base?:

```
Derivada(int num) : Base(num) { ... }
```

1. Para delegar el constructor usamos la member initializer list.
2. En C++ el constructor de Base se llama antes que el constructor de Derivada.
3. Con los destructores es al revés (como una pila).

Tipos:

	Tipo de herencia de la clase Derivada		
Método de la clase Base	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	No accesible	No accesible	No accesible

Polimorfismo

¿Qué es el Polimorfismo?

1. Tener varias formas.
2. En C++, significa que la misma llamada a función tiene distintos comportamientos dependiendo del tipo del objeto.
3. No queremos que dependa del tipo de la variable.

Polimorfismo en C++

- Por defecto, C++ resuelve a qué método llamar en tiempo de compilación.
- A esto se lo conoce como static linkage, static resolution, o early binding.
- Para que esto se resuelva en tiempo de ejecución debemos usar el modificador virtual.
- En Java todo es virtual por defecto, y para que no sea virtual hay que usar final.
- Usar virtual crea VTables, que sirven para la resolución dinámica, lo cual empeora la performance.
- A esto se lo conoce como dynamic linkage, o late binding.

```
1 #include <iostream>
2
3 class Base {
4 public:
5     void metodo() {
6         std::cout << "Base\n";
7     }
8 };
9
10 class Derivada1: public Base {
11 public:
12     void metodo() {
13         std::cout << "Derivada_1\n";
14     }
15 };
16
17 class Derivada2: public Base {
18 public:
19     void metodo() {
20         std::cout << "Derivada_2\n";
21     }
22 }
```

```
25 int main(void) {
26     Base *ptr;
27     Derivada1 d1;
28     Derivada2 d2;
29
30     ptr = &d1;
31     ptr->metodo();
32
33     ptr = &d2;
34     ptr->metodo();
35
36     return 0;
37 }
```

La salida será:
Base
Base

```
1 #include <iostream>
2
3 class Base {
4 public:
5     virtual void metodo() {
6         std::cout << "Base\n";
7     }
8 };
9
10 class Derivada1: public Base {
11 public:
12     virtual void metodo() {
13         std::cout << "Derivada_1\n";
14     }
15 };
16
17 class Derivada2: public Base {
18 public:
19     virtual void metodo() override {
20         std::cout << "Derivada_2\n";
21     }
22 }
```

```
25 int main(void) {
26     Base *ptr;
27     Derivada1 d1;
28     Derivada2 d2;
29
30     ptr = &d1;
31     ptr->metodo();
32
33     ptr = &d2;
34     ptr->metodo();
35
36     return 0;
37 }
```

La salida será:
Derivada 1
Derivada 2

- Notar que la variable ptr siempre es de tipo Base*, lo que cambia es el tipo del objeto.
- Esto nos permite, por ejemplo, tener una lista de punteros a Base, e iterarla llamando siempre a metodo(), sin depender del tipo del objeto en tiempo de compilación.
- El identificador override (C++11) obliga a que la firma coincida con la del método de la clase base.

Clases abstractas e Interfaces

- No se puede instanciar una clase abstracta.
- En C++ no existe un modificador abstract, una clase abstracta es la que tiene un método virtual puro.
- La única manera de hacer interfaces en C++ es usando clases abstractas sin atributos.

```

1 #include <iostream>
2
3 class Base {
4 public:
5     virtual void metodo() = 0;
6 };
7
8 class Derivada1: public Base {
9 public:
10    virtual void metodo() {
11        std::cout << "Derivada_1\n";
12    }
13 };
14
15 class Derivada2: public Base {
16 public:
17    virtual void metodo() {
18        std::cout << "Derivada_2\n";
19    }
20 };
21
22
23 int main(void) {
24     // no compila!
25     Base base;
26     Derivada1 d1;
27     Derivada2 d2;
28
29     return 0;
30 }

```

Destructores virtuales

Cuando el objeto es de tipo Derivada (aunque la variable sea un Base*) se llamará a:

1. Constructor de Base
2. Constructor de Derivada
3. Destructor de Derivada
4. Destructor de Base

Conceptos Importantes:

Static linkage / early binding → el llamado al método se resuelve en tiempo de compilación (según la variable).

Dynamic linkage / late binding → el llamado al método se resuelve en tiempo de ejecución (según el objeto en memoria).

Problem: Object Slicing!

Ocurre cuando, por ejemplo, trabajamos con una clase base y una clase derivada, si instanciamos una clase derivada con ciertos datos y luego una clase base que apunte a la derivada, vamos a tomar los datos de Derivada que pertenezcan a la clase Base, “cortando” el resto.

Cómo puedo evitar esto? → Se deben eliminar el constructor por copia y la asignación por copia, de forma tal que no haya lugar a errores como este.

```

class A {
    int a;
public:
    A(int a) : a(a) {}
    void print() {
        std::cout << "A._a:" <<
            a << std::endl;
    }
    int getA() {
        return a;
    }
};

class B : public A {
    int b;
public:
    B(int a, int b) : A(a), b(b) {}
    void print() {
        std::cout << "B._a:" <<
            getA() << "._b:" <<
            b << std::endl;
    }
};

24 int main() {
25     A var1(1);
26     var1.print();
27
28     B var2(2, 3);
29     var2.print();
30
31     A& var3 = var2;
32     var3.print();
33
34     var3 = var1;
35     var3.print();
36     var2.print();
37     return 0;
38 }

```

Ejercicios

3) Explique qué son **los métodos virtuales** y para qué sirven. De un breve ejemplo donde su uso sea imprescindible.

Los métodos virtuales son métodos cuyo llamado se resuelve en tiempo de ejecución (según el objeto en memoria), esto permite que pueda suceder el polimorfismo (en C++, el llamado a la función tiene distintos comportamiento según el tipo de objeto).

A esto se lo conoce como Dynamic Linkage / Late Binding y es necesario declarar el método con el modificador “virtual”, ya que de otra forma el llamado se resolvería en tiempo de compilación (según la variable), evitando que suceda el polimorfismo.

Un ejemplo de uso imprescindible es:

```
class Base {
public:
    virtual void print() { std::cout << "soy clase base" << std::endl;
};

class Derivada : public Base {
public:
    void print() { std::cout << "soy clase derivada" << std::endl;
};

int main(){
    Base *ptr = new Derivada();
    ptr->print();
    delete ptr;
    return 0;
}
```

En caso de no declarar el destructor de “Base” con el modificador “virtual”, si bien se llama al constructor de Base y después al de Derivada, al hacer “delete”, solo se llamaría al destructor de Base, generando así un leak de memoria.

Sobrecarga de Operadores

Tipos de operadores:

- Aritméticos: +, -, *, /, %, ++, --
- Lógicos: &&, ||, !
- Comparación: <, >, <=, >=, ==
- Bitwise: &, |, <<, >>, ~, ^
- Asignación: =, +=, -=, *=, /=, Etc.
- Misceláneos
 - Punteros y referencias: [], *, ->, &
 - Conversión de tipos: (int), (float), Etc. (casteo)
 - Invocación de funciones: ()

Tipos según argumentos:

- Unarios: i++;
- Binarios: i == j
- Ternarios: i < 0 ? 0 : i
- N-Arios
 - No empleados para tipos nativos en C/C++
 - Ej: arreglo[i, j, k, l, m]

Sobrecarga de Operadores en C++

- Aplica solamente a operaciones que utilicen clases/structs
- Permite sobrecargar los operadores default de clases/structs
- Utiliza el comportamiento del compilador para convertir "funciones de operador"

Ej. Unario:

var1.operator=(var2) //equivale a: var1=var2

Ej. Binario:

var1 = operator+(var2, var3) //equivale a: var1=var2+var3

Ejercicios

5) **Declare** una clase de elección libre. Incluya todos los **campos de datos** requeridos con su **correcta exposición/publicación**, y los **operadores ++, -, ==, >> (carga), << (impresión), constructor move y operador float()**.

```
class Punto(){
private:
    float x;
    float y;

public:
    Punto() : x(0), y(0) {}

    Punto(float x, float y) : x(x), y(y) {}

    Punto(const Punto& other) { // Constructor de copia
        this->x = other.x;
        this->y = other.y;
    }

    Punto(Punto&& other){      // Constructor por mov
        this->x = other.x;
        this->y = other.y;
    }

    Punto& operator++() { //Pre-Incremento
        ++x;
        ++y;
        return *this;
    }

    Punto operator++(int) { //Post-Incremento
        Complex copy(*this);
        ++x;
        ++y;
        return copy;
    }

    bool operator==(const Punto& other) const {
        return this->x == other.x && this->y == other.y;
    }

    friend Punto operator+(const Punto& p1, const Punto& p2);
    friend Punto operator-(const Punto& p1, const Punto& p2);
    friend istream& operator>>(istream& in, Punto& p);
    friend ostream& operator<<(ostream& out, const Punto& p);
```

```

    operator float() const {
        return sqrt(x*x + y*y);
    }

}

...
Punto operator+(const Punto& p1, const Punto& p2) {
    return Punto(p1.x + p2.x, p1.y + p2.y);
}

Punto operator-(const Punto& p1, const Punto& p2) {
    return Punto(p1.x - p2.x, p1.y - p2.y);
}

istream& operator>>(istream& in, Punto& p) {
    in >> p.x >> p.y;
    return in;
}

ostream& operator<<(ostream& out, const Punto& p) {
    out << "(" << p.x << ", " << p.y << ")";
    return out;
}

```

Una clase friend puede acceder a los atributos y métodos privados.
Es posible hacer friend no solo a una clase sino a un solo método.

Programación Orientada a Eventos

Paradigmas de programación

- Programación secuencial: Se basa en definir la secuencia de pasos que el programa debe seguir
- POO: SE abstraen las entidades del problema, su comportamiento, su estado y sus interrelaciones.
- Programación Funcional: Las funciones se utilizan como predicados matemáticos.
- POE: Se define cómo reaccionar a distintos sucesos.

¿En qué consiste POE?

1. Toda **acción** que ejecute el sistema será en respuesta a los **sucesos** que acontezcan.
2. A esos **sucesos** los llamamos **eventos**
3. Las **acciones a ejecutar**, lo que programamos, son los **manejadores**
4. A las fuentes de eventos las vamos a nombrar **actores**

¿De dónde vienen los eventos? → Acciones de usuario, basados en tiempo, generados por otro evento, definidos por el usuario, eventos del entorno (hotplug)

Event Queue

Usamos una cola de eventos para poder proteger el orden de la ejecución de los eventos en un programa concurrente. Proteger la cola nos brinda mucha seguridad respecto a la concurrencia (pero no nos salva).

¿Quién lee los eventos de la cola? Event Loop

Al bucle que lee los eventos de la cola se lo llama event loop, que es muy similar al game loop que se usa en los juegos para simular el paso del tiempo

Handlers/Listeners

- Los **handlers** son secciones de código que saben cómo responder a la aparición de un **evento**.
- Vamos a unir (**bind**) los **handlers** a **eventos**
- Cuando hay GUI es importante hacer **handlers cortos** y delegar operaciones largas en otros threads (si no hay GUI puede no ser tan importante)

Ejercicios

- 6) **Describa el concepto de loop de eventos (events loop) utilizado en programación orientada a eventos y, en particular, en entornos de interfaz gráfica (GUIs).**

El event loop es el loop, es decir la repetición, que procesa la cola de eventos. Es muy similar al game loop que se usa en los juegos para simular el paso del tiempo. Es importante destacar que se recomienda fuertemente que los handlers de cada evento no se bloqueen o realicen tareas que lleven mucho tiempo con el fin de no bloquear el programa y que el event loop pueda procesar correctamente la cola de eventos.

Namespaces, friends and Smart Pointers

- **namespace** → Los namespaces (espacios de nombres) proporcionan un método para evitar conflictos de nombres en proyectos grandes.
 - Los símbolos declarados dentro de un bloque namespace se colocan en un ámbito con nombre que evita que se confundan con símbolos con nombres idénticos en otros ámbitos.
- **using** → Introduce un nombre que se definió en otro lugar, en la región declarativa donde aparece

Friend

Una clase **friend** puede acceder a los atributos y métodos privados.

Es posible hacer **friend** no solo a una clase sino a un solo método.

Friend no es transitivo: el amigo de mi amigo no es necesariamente mi amigo.

Smart pointers: Concepto

- **std::unique_ptr** → es un puntero inteligente que posee y administra otro objeto a través de un puntero. Solo un objeto **unique_ptr** puede poseer el objeto y elimina ese objeto cuando el **unique_ptr** queda fuera de alcance (RAII)
- **std::shared_ptr** → es un puntero inteligente que posee y administra otro objeto a través de un puntero. Varios objetos **shared_ptr** pueden poseer el mismo objeto. El objeto es destruido cuando el último objeto pierde la referencia (RAII)
- **std::weak_ptr** → es un puntero inteligente que contiene una referencia no propietaria ("débil") a un objeto administrado por **std::shared_ptr**. Debe convertirse a **std::shared_ptr** para acceder al objeto al que se hace referencia.

Ejercicios

2) Explique **qué es** cada uno de los siguientes, haciendo referencia a su **inicialización**, su **comportamiento** y el **area de memoria** donde residen:

- a) Una variable **global static**
 - b) Una variable **local static**
 - c) Un **atributo de clase static**.
- a. **global static**: Es una variable declarada fuera de cualquier función que tiene el alcance limitado al archivo donde fue definida por la especificación **static**.
 - i. **Inicialización**: Se inicializa una sola vez cuando se carga el programa en memoria. Fuera de cualquier función.
 - ii. **Comportamiento**: Al ser **static** no puede ser accedida desde otros archivos fuente. La duración de esta variable es la misma que la del programa.
 - iii. **Memoria**: Data Segment
 - b. **local static**: Es una variable declarada dentro de cualquier función o bloque cuya duración es estática por la especificación **static**.
 - i. **Inicialización**: Se inicializa una sola vez cuando se la instancia. Dentro de cualquier función.
 - ii. **Comportamiento**: En lugar de ser creada y destruida cada vez que se llama a la función (su scope), permanece en memoria durante la vida del programa.

- iii. Memoria: Data Segment
- c. atributo de clase `static`: Es un atributo compartido por todas las instancias de una clase
 - i. Inicialización: Debe definirse fuera de la clase e inicializarse explícitamente.
 - ii. Comportamiento: Su valor persiste durante toda la ejecución del programa y a su vez es compartido por todas las instancias de la clase, por ende si una instancia lo modifica, esto se verá reflejado en las otras.
 - iii. Memoria: Data Segment

Templates

Templates: un único código para gobernarlos a todos

- La idea es similar a la alternativa del precompilador: usar el mismo código pero reemplazando el tipo particular int/char por uno genérico T.
- Pero a diferencia del precompilador, el código template es procesado por el compilador: es más seguro, hay chequeo de tipos y los errores están mejor explicados (bueno, hasta cierto punto)
- No solo las clases pueden ser templates, los struct y las funciones también.
- Se puede parametrizar por tipo (class T) o por una constante (int size).
- Como todo parámetro, pueden tener un default.
- Al llamar a una función template podemos especificar sobre que tipos estamos trabajando o podemos dejar que el compilador lo deduzca automáticamente basándose en los parámetros.
- Tener en cuenta que la deducción automática puede no tener el efecto que uno quiere pues no siempre es fácil determinar el tipo de los parámetros: el número 3 es un int o un char?

Importante:

- Jamás implementar un template al primer intento. Crear una clase prototipo (Array_ints, testarla y luego pasarla a template Array<T>)
- Si se va a usar el templates con punteros, evitar el code bloat implementando la especialización void* (Array<void*>) y luego la especialización parcial T* (Array<T*>).
- Opcionalmente, implementar especializaciones optimizadas (Array<bool>)

Tipos de templates:

- *Especializado*: Esto permite tener código eficiente para tipos particulares, es decir implementaciones más apropiadas. (ejemplo tener Array<T> y Array<Bool>, Se requiere implementar primero Array<T> antes de la especialización.)

```
template<typename T>
bool cmp(T a, T b){
    return a == b;
}
template <>
bool cmp(const char *a, const char *b){
    return strcmp(a, b, SZ);
}
```

- Parcialmente especializado: Cuando se quiera usar un array de punteros, se utilizara el código especializado para ellos (Array<T*>), la especialización parcial utiliza a la completa para void*. al utilizar este código el compilador no genera código por ende se evita el code bloat, primero se debe implementar Array<T> y Array<void *>

```
template <typename T>
Class Array<T*>:private Array<void*>{
public:
    void set(int p, T* v){
        Array<void*>::set(p, v);
    }
    T* get(int p){ Array<void*>::get(p); }
}
```

Ejercicios

- 9) Implemente una función **C++** denominada **Sacar** que reciba dos listas de elementos y devuelva una nueva lista con los elementos de la primera que no están en la segunda:
- ```
std::list<T> Sacar(std::list<T> a, std::list<T> b);
```

```
template <typename T>
std::list<T> Sacar(std::list<T> a, std::list<T> b){
 std::list<T> resultado;
 for(const auto& elem : a) {
 if(std::find(b.begin(), b.end(), elem) == b.end()){
 resultado.push_back(elem);
 }
 }
 return resultado;
}
```

- 10) Implemente una función **C++** denominada **DobleSiNo** que reciba dos listas de elementos y devuelva una nueva lista **duplicando los elementos de la primera que no están en la segunda**:

```
std::list<T> DobleSiNo(std::list<T> a, std::list<T> b);
```

```
template <typename T>
std::list<T> DobleSiNo(std::list<T> a, std::list<T> b){
 std::list<T> resultado;
 for(const auto& elem : a){
 if(std::find(b.begin(), b.end(), elem) == b.end()){
 resultado.push_back(elem);
 resultado.push_back(elem);
 }else{
 resultado.pushback(elem);
 }
 }
 return resultado;
}
```

Explicación de: `if (std::find(b.begin(), b.end(), elem) == b.end())`  
`std::find` busca el elemento `elem` dentro del rango `[b.begin(), b.end())`.  
Si `std::find` no encuentra el elemento en la lista `b`, devuelve `b.end()`.  
Por lo tanto, esta condición `(== b.end())` *verifica que el elemento no está en b*.

## Ejercicios de Final

6) ¿Qué es una **macro** de C? Describa las **buenas prácticas** para su definición y **ejemplifique**.

Una macro en C es definida por una directiva del preprocesador y se utiliza para realizar sustituciones en el código fuente antes de la compilación. Las macros son útiles para definir constantes, simplificar fragmentos repetitivos o habilitar/deshabilitar partes del código según ciertas condiciones. Existen distintos tipos de macros:

→ *Reemplazo simple*: Para definir constantes o nombres simbólicos reemplazados en el código.

Ej: `#define PI 3.14`

→ *Macros con parámetros*: Actúan como plantillas, pero no verifican tipos ni respetan las reglas del compilador para funciones.

Ej: `#define SQUARE(x) ((x) * (x))`

→ *Macros condicionales*: Permite incluir o excluir secciones de código en tiempo de compilación.

Ej:

```
#define DEBUG
int main() {
#ifdef DEBUG
 std::cout << "Debugueando" << std::endl;
#else
 std::cout << "No debugueo" << std::endl;
#endif
 return 0;
}
```

*Buenas prácticas para usar macros:*

- Usar nombres descriptivos
- Escribir los nombres de las macros en mayúsculas
- Declararlas al principio del archivo.
- Usar paréntesis en las de parámetros para evitar errores inesperados en las operaciones.
- Considerar alternativas como `const` para valores constantes y funciones `inline` para operaciones, ya que son más seguras y evitan errores típicos de las macros.

8) Describa el **proceso de transformación de código** fuente a un ejecutable. Precise las **etapas**, las **tareas desarrolladas** y los **tipos de error** generados en cada una de ellas.

*Proceso de transformación de código fuente a un ejecutable:*

- *Preprocesador*: Expande macros, incluye bibliotecas y procesa directivas (`#include`, `#define`, `#ifdef`, etc.), además de eliminar comentarios. El código resultante aún está en C y se guarda como un archivo con extensión `.i`
- *Compilación*: Traduce el código del archivo `.i` a lenguaje Assembly, generando un archivo `.s`. En este paso, el compilador verifica sintaxis y semántica.
- *Ensamblador*: Traduce el archivo `.s` (Assembly) a un archivo de código máquina, generando un archivo objeto `.o`. Este archivo es reubicable, lo que significa que

todavía no está listo para ejecutarse, pero sí contiene instrucciones binarizadas que el CPU entiende.

→ *Linker*. Une todos los archivos .o generados por el ensamblador junto con las bibliotecas necesarias (estáticas o dinámicas) para producir un archivo ejecutable final.

4) ¿Qué es un **functor**? ¿Qué ventaja ofrece frente a una función convencional? **Ejemplifique**.

Un functor es una clase que sobrecarga el operador `operator()`. Esto permite que las instancias de esa clase sean invocadas como si fueran funciones.

La ventaja principal de un functor por sobre una función es:

**Estado interno:** Los funtores pueden mantener un estado interno a través de sus atributos. Esto permite personalizar su comportamiento entre llamadas, algo que no es posible con funciones normales.

```
class Sumador {
private:
 int suma;
public:
 explicit Sumador(int num) : suma(num){}

 void operator()(int value) {
 suma += value;
 }

 int getSuma() const {
 return suma;
 }
};

int main() {
 Sumador sumador(0);

 int numbers[] = {1, 2, 3}

 for (int i : numbers) {
 sumador(i);
 }

 std::cout << "La suma: "
 sumador.getSuma() <<
 std::endl;

 return 0;
}
```

5) Explique qué se entiende por “**compilación condicional**”. **Ejemplifique** mediante código.

La compilación condicional es una técnica que permite incluir o excluir partes del código durante la compilación, dependiendo de ciertas condiciones o definiciones. Esto se utiliza comúnmente para realizar ajustes en el código para diferentes plataformas, configuraciones de compilación o entornos sin modificar manualmente el código fuente cada vez. En C/C++, esto se maneja mediante directivas del preprocesador, como `#ifdef`, `#ifndef`, `#else`, `#endif`. **MACROS!**

Ejemplo:

```
int main() {
 #ifdef _WIN32 // Detecta Windows
 printf("Ejecutando en Microsoft Windows\n");
 #elif defined(__linux__) // Detecta Linux
 printf("Ejecutando en Linux\n");
 #else
 printf("Sistema operativo no reconocido\n");
 #endif
 return 0;
}
```

7) Escriba las siguientes definiciones/declaraciones:

- a) Definición de una la función SUMA, que tome dos enteros largos con signo y devuelva su suma. Esta función sólo debe ser visible en el módulo donde se la define.
- b) Declaración de un puntero a puntero a entero sin signo.
- c) Definición de un caracter solamente visible en el módulo donde se define.

a. static long int SUMA(long int a, long int b) { CLAVE: STATIC  
    return a + b;  
}  
b. unsigned int \*\*ptr;  
c. static char caracter = 'A';

8) ¿Qué valor arroja sizeof(int)? Justifique .

El operador sizeof devuelve el tamaño de un tipo de dato en bytes, que puede variar dependiendo de la arquitectura del computador. En las arquitecturas de 32-bit y 64-bit el tamaño de un int es 4 bytes.

5) Escriba el .H de una biblioteca de funciones ISO C para cadenas de caracteres. Incluya, al menos, 4 funciones.

```
#ifndef CADENA_H
#define CADENA_H

#include <stdbool.h>

bool MasDeUnaPalabra(const char * cadena);

int longitudCadena(const char * cadena);

bool utilizaTodoElAbecedario(const char * cadena);

void copiarCadena(char * c1, const char * c2);

#endif CADENA_H
```

## Ejercicios Archivos

4) Escribir **un programa ISO C** que reciba por argumento el nombre de un archivo de texto y lo procese sobre sí mismo (sin crear archivos intermedios ni subiendo todo su contenido a memoria). El procesamiento consiste en **eliminar las líneas de 1 sola palabra**.

```
int main() {
 FILE *archivo = fopen("archivo.txt", "r+");

 char linea[1024];
 long read = 0, write = 0;

 while (fgets(linea, sizeof(linea), archivo) != NULL) {
 write = ftell(archivo);

 if (strchr(linea, ' ') != NULL) {
 fseek(archivo, read, SEEK_SET);
 fputs(linea, archivo);
 write = ftell(archivo);
 }

 fseek(archivo, read, SEEK_SET);
 }

 ftruncate(fileno(archivo), write);
 fclose(archivo);
 return 0;
}
```

### Desarrollo:

1. Abrir el archivo con: `fopen("archivo.txt", "r+")` → para lectura y escritura
2. Leer cada línea del archivo: `fgets(linea, sizeof(linea), archivo)`
3. Obtener la posición del puntero de archivo: `write = ftell(archivo)`  
`ftell(archivo)` devuelve la posición actual del puntero de lectura/escritura en el archivo, es decir, el byte en el que se encuentra en ese momento.
4. Mover el puntero con `fseek(archivo, read, SEEK_SET)`  
Mueve el puntero del archivo a la posición indicada por `read` (una variable que guarda una posición previa en el archivo). `SEEK_SET` especifica que el desplazamiento se cuenta desde el inicio del archivo
5. Escribir en el archivo con `fputs`: `fputs(linea, archivo)`  
`fputs(linea, archivo)` escribe la línea almacenada en el arreglo `linea` en la posición actual del puntero de archivo. Después de escribir, el puntero se mueve automáticamente hacia adelante, más allá del contenido escrito
6. Actualizar la posición de escritura: `write = ftell(archivo)`  
Después de escribir la línea, `ftell(archivo)` se llama nuevamente para obtener la nueva posición del puntero de escritura, que se guarda en `write`
7. Restablecer el puntero a la posición `fseek(archivo, read, SEEK_SET)`

3) Escribir un programa ISO C que procese el archivo "nros2bytes.dat" sobre sí mismo, duplicando los enteros de 2 bytes múltiplos de 3.

```
int main() {
 FILE* file = fopen("nros2bytes.dat", "r+b");
 if (file == NULL) {
 return -1;
 }

 int read = 0, write = 0;
 int16_t num;

 while (fread(&num, sizeof(int16_t), 1, file) == 1) {
 if (num % 3 == 0) {
 num *= 2;
 }

 fseek(file, write, SEEK_SET);
 fwrite(&num, sizeof(int16_t), 1, file);
 write += sizeof(int16_t);

 read += sizeof(int16_t);
 fseek(file, read, SEEK_SET);
 }

 fflush(file);

 if (ftruncate(fileno(file), write) != 0) {
 printf("Error al truncar el archivo.\n");
 }

 fclose(file);
 return 0;
}
```

Desarrollo:

1. Abrir: `fopen("archivo.dat", "r+b")` → lectura y escritura BINARIOS
2. Lectura: `while (fread(&num, sizeof(int16_t), 1, file) == 1)`
  - a. Se lee un número de 2 bytes (`int16_t`) del archivo.
  - b. Si `fread` devuelve 1, significa que se leyó un número correctamente.
3. Escritura en Write:
  - a. `fseek(file, write, SEEK_SET)`: mueve el puntero de escritura a la posición `write`.
  - b. `fwrite(&num, sizeof(int16_t), 1, file)`: escribe `num` en esa posición.
4. Actualizar los índices
  - a. `write += sizeof(int16_t)`: `write` avanza 2 bytes porque acabamos de escribir un número de 2 bytes.



- b. `read += sizeof(int16_t)`: `read` avanza 2 bytes para leer el siguiente número.
  - c. `fseek(file, read, SEEK_SET)`: mueve el puntero de lectura a la nueva posición.
- 5. Truncar con **write**
- 6. Cerrar los archivos

### Funciones de Archivos:

| Función             | Descripción                                             | Sintaxis                                                                              | Parámetros                                                                                                                                                                                                                                                                   | Utilidad / Uso                                                                  |
|---------------------|---------------------------------------------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <code>fopen</code>  | Abre un archivo en un modo específico.                  | <code>FILE* fopen(const char* filename, const char* mode);</code>                     | <ul style="list-style-type: none"> <li>- <code>filename</code>: Nombre del archivo.</li> <li>- <code>mode</code>: Modo de apertura ("<code>r</code>", "<code>w</code>", "<code>a</code>", "<code>rb</code>", etc.).</li> </ul>                                               | Abre archivos para lectura, escritura o actualización.                          |
| <code>fclose</code> | Cierra un archivo previamente abierto.                  | <code>int fclose(FILE* stream);</code>                                                | <ul style="list-style-type: none"> <li>- <code>stream</code>: Puntero al archivo.</li> </ul>                                                                                                                                                                                 | Libera recursos y guarda cambios en el archivo.                                 |
| <code>fread</code>  | Lee datos binarios desde un archivo.                    | <code>size_t fread(void* ptr, size_t size, size_t count, FILE* stream);</code>        | <ul style="list-style-type: none"> <li>- <code>ptr</code>: Dirección donde se almacenarán los datos.</li> <li>- <code>size</code>: Tamaño de cada elemento.</li> <li>- <code>count</code>: Cantidad de elementos a leer.</li> <li>- <code>stream</code>: Archivo.</li> </ul> | Lee datos de un archivo binario en bloques de un tamaño fijo.                   |
| <code>fwrite</code> | Escribe datos binarios en un archivo.                   | <code>size_t fwrite(const void* ptr, size_t size, size_t count, FILE* stream);</code> | <ul style="list-style-type: none"> <li>- <code>ptr</code>: Dirección de los datos a escribir.</li> <li>- <code>size</code>: Tamaño de cada elemento.</li> <li>- <code>count</code>: Cantidad de elementos a escribir.</li> <li>- <code>stream</code>: Archivo.</li> </ul>    | Guarda datos binarios en un archivo.                                            |
| <code>fseek</code>  | Mueve el puntero del archivo a una posición específica. | <code>int fseek(FILE* stream, long offset, int origin);</code>                        | <ul style="list-style-type: none"> <li>- <code>stream</code>: Archivo.</li> <li>- <code>offset</code>: Cantidad de bytes a mover.</li> <li>- <code>origin</code>: Posición de referencia (<code>SEEK_SET</code>, <code>SEEK_CUR</code>, <code>SEEK_END</code>).</li> </ul>   | Controla la posición de lectura/escritura dentro del archivo.                   |
| <code>ftell</code>  | Retorna la posición actual del puntero del archivo.     | <code>long ftell(FILE* stream);</code>                                                | <ul style="list-style-type: none"> <li>- <code>stream</code>: Archivo.</li> </ul>                                                                                                                                                                                            | Útil para saber en qué posición del archivo está el puntero.                    |
| <code>rewind</code> | Reinicia el puntero del archivo al inicio.              | <code>void rewind(FILE* stream);</code>                                               | <ul style="list-style-type: none"> <li>- <code>stream</code>: Archivo.</li> </ul>                                                                                                                                                                                            | Devuelve el puntero al inicio del archivo sin necesidad de <code>fseek</code> . |

|                        |                                                             |                                                                  |                                                                                                                                                         |                                                                                    |
|------------------------|-------------------------------------------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <code>fflush</code>    | Fuerza la escritura de datos en el archivo.                 | <code>int fflush(FILE* stream);</code>                           | - <code>stream</code> : Archivo. (Si es <code>NULL</code> , vacía todos los buffers de salida).                                                         | Útil para asegurar que los datos se escriban inmediatamente en disco.              |
| <code>ftruncate</code> | Corta o extiende el tamaño de un archivo.                   | <code>int ftruncate(int fd, off_t length);</code>                | - <code>fd</code> : Descriptor de archivo.<br>- <code>length</code> : Nuevo tamaño del archivo.                                                         | Elimina bytes sobrantes o expande un archivo a un tamaño específico.               |
| <code>fileno</code>    | Obtiene el descriptor de archivo de un <code>FILE*</code> . | <code>int fileno(FILE* stream);</code>                           | - <code>stream</code> : Archivo.                                                                                                                        | Necesario para funciones como <code>ftruncate</code> .                             |
| <code>fprintf</code>   | Escribe texto formateado en un archivo.                     | <code>int fprintf(FILE* stream, const char* format, ...);</code> | - <code>stream</code> : Archivo.<br>- <code>format</code> : Cadena con formato ("%d %s").<br>- Otros valores según el formato.                          | Similar a <code>printf</code> , pero escribe en un archivo en lugar de la consola. |
| <code>fscanf</code>    | Lee texto formateado desde un archivo.                      | <code>int fscanf(FILE* stream, const char* format, ...);</code>  | - <code>stream</code> : Archivo.<br>- <code>format</code> : Cadena con formato ("%d %s").<br>- Punteros a variables donde almacenar los valores leídos. | Similar a <code>scanf</code> , pero lee desde un archivo.                          |
| <code>feof</code>      | Verifica si se alcanzó el final de un archivo.              | <code>int feof(FILE* stream);</code>                             | - <code>stream</code> : Archivo.                                                                                                                        | Útil para saber si se llegó al final de un archivo durante la lectura.             |
| <code>ferror</code>    | Verifica si hubo un error en operaciones de archivo.        | <code>int ferror(FILE* stream);</code>                           | - <code>stream</code> : Archivo.                                                                                                                        | Ayuda a detectar errores en la manipulación de archivos.                           |